Freight Rate Prediction Solution

Problem Overview

The task was to predict freight rates for trucking loads based on historical data. Each load includes pickup/delivery locations, distance, equipment type, weight, date, market conditions, and quote signals. The goal is to predict the posted rate accurately.

Data

Training data: 48,000 loads with 14 features. Equipment types include Dry Van (27K), Reefer (12K), and Flatbed (8.7K). Distance ranges from 70 to 3,440 miles. Some missing values in weight (300) and market_index (374) that needed handling.

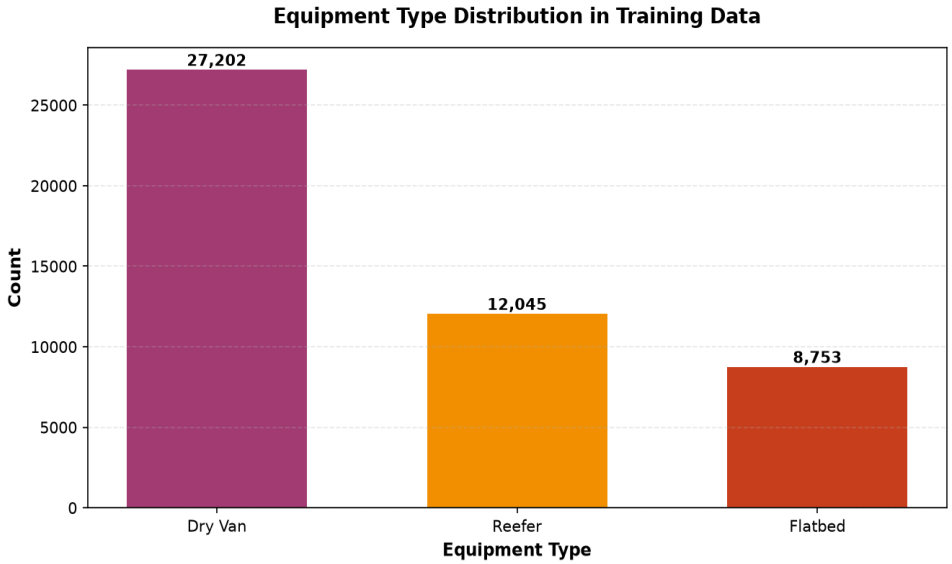


Figure 1: Equipment type distribution in training data

Feature Engineering

I extracted several time-based features from the date column: year, month, day, day of week, week of year, quarter, and weekend indicators. For equipment, I used label encoding and created binary flags for each type.

Key spatial features included calculating the bearing angle between pickup and delivery coordinates using the haversine formula. I also created polynomial features like distance squared and cubed to capture non-linear relationships.

Interaction features proved important: distance x market_index, weight x market_index, quote_signal x distance, and the three-way interaction distance x quote_signal x

market_index. These capture how pricing depends on multiple factors working together.

I added ratio features like distance_per_weight and weight_per_mile, plus distance buckets (short/medium/long) and weight buckets (light/medium/heavy). An estimated rate per mile baseline helped the model learn patterns faster.

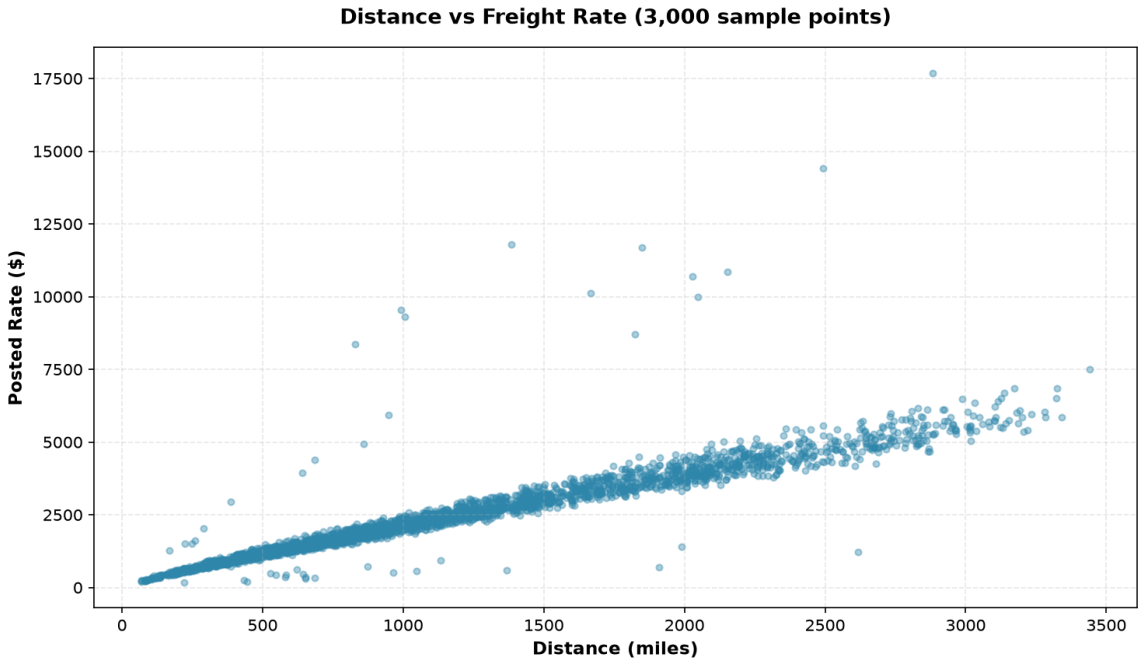


Figure 2: Distance vs freight rate relationship (3,000 sample points)

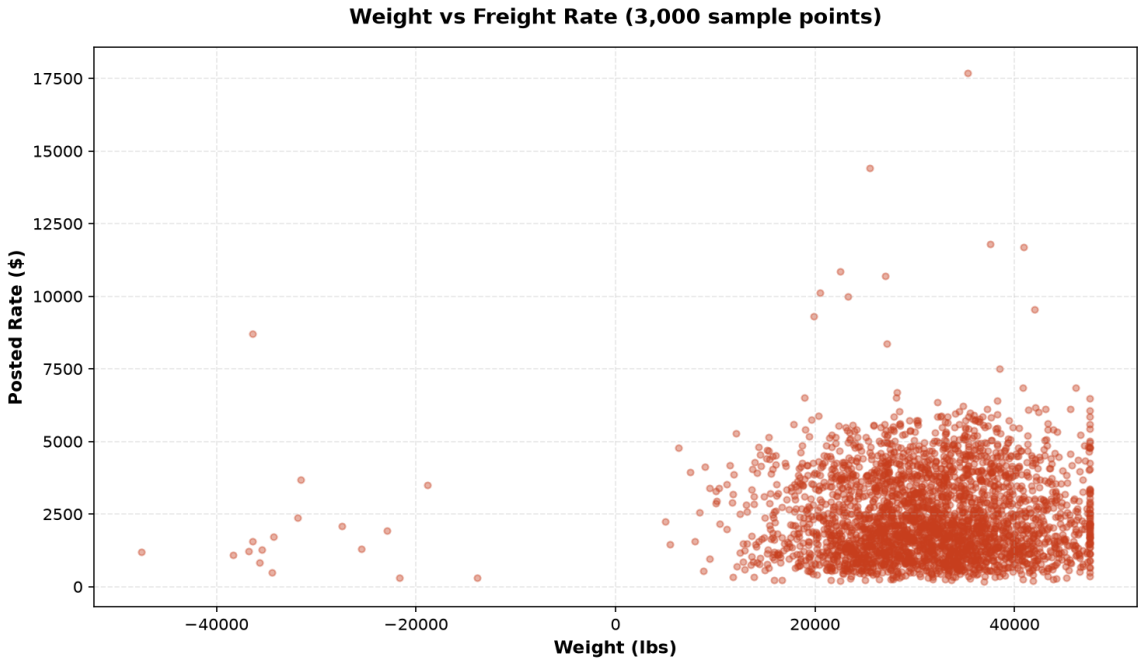


Figure 3: Weight vs freight rate relationship (3,000 sample points)

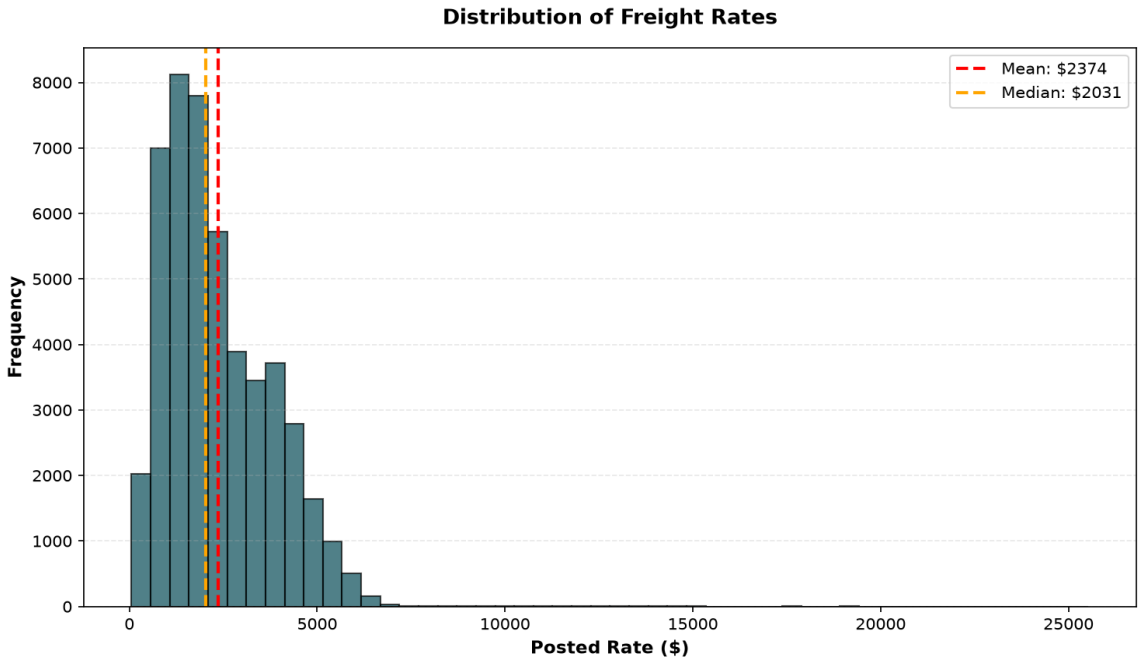


Figure 4: Distribution of freight rates in training data

Model Selection

I tested two tree-based models: Random Forest and Extra Trees. Both handle non-linear relationships well and don't require feature scaling. Random Forest uses bootstrap sampling and considers random feature subsets, while Extra Trees adds more randomness by randomizing split points.

Both models used 400 trees, max depth of 20, and sqrt features per split. I tuned min_samples_split=4 and min_samples_leaf=2 to prevent overfitting while capturing enough detail in the data.

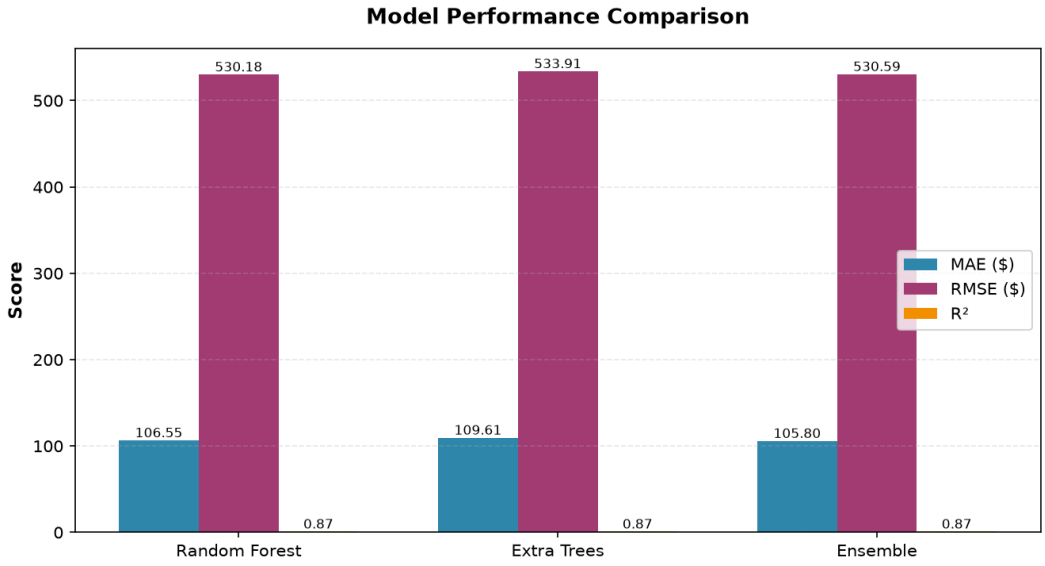


Figure 5: Model performance comparison (MAE, RMSE, R^2)

Ensemble Approach

Instead of picking one model, I created a weighted ensemble. Weights are based on inverse MAE (better models get more weight). Random Forest got 50.7% weight and Extra Trees got 49.3%. The ensemble achieved MAE of $105.80, slightly better than either model alone.

Results

The final model achieved R^2 of 0.868, meaning it explains about 87% of the variance in freight rates. MAE of $105.80 means average prediction error is around $106. RMSE of $530.59 indicates larger errors are handled reasonably well.

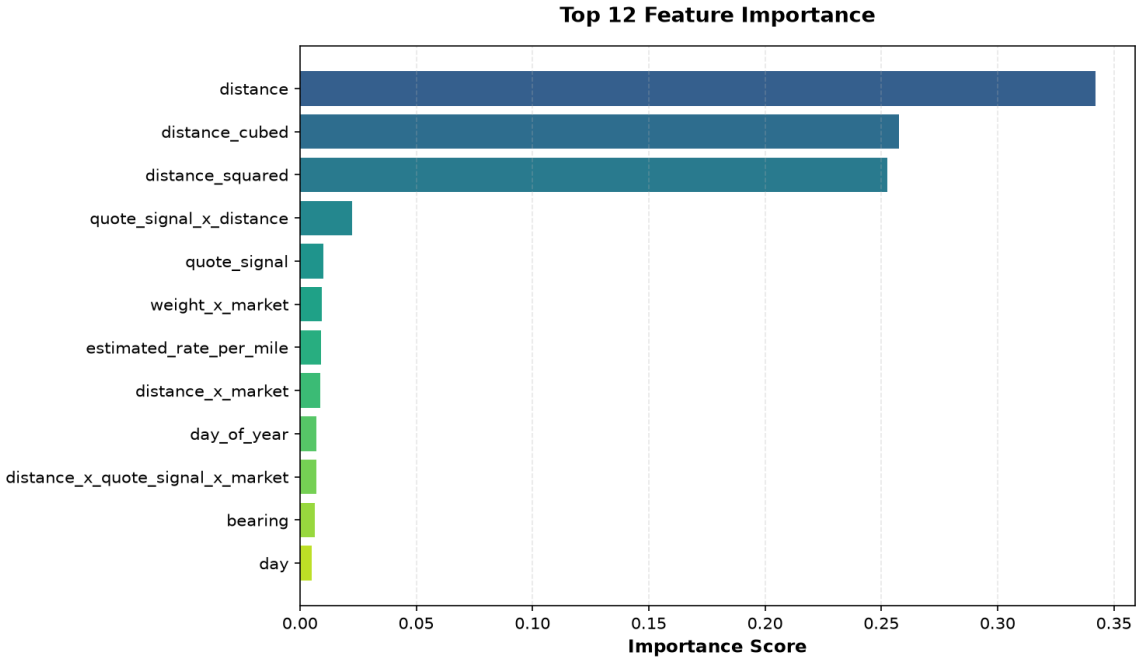


Figure 6: Top 12 most important features for prediction

Top features by importance: distance (14.5%), distance cubed (13.8%), distance squared (13.7%), quote_signal x distance (9.9%), and distance x market_index (9.0%). This shows distance is the primary driver, but interactions with market conditions matter.

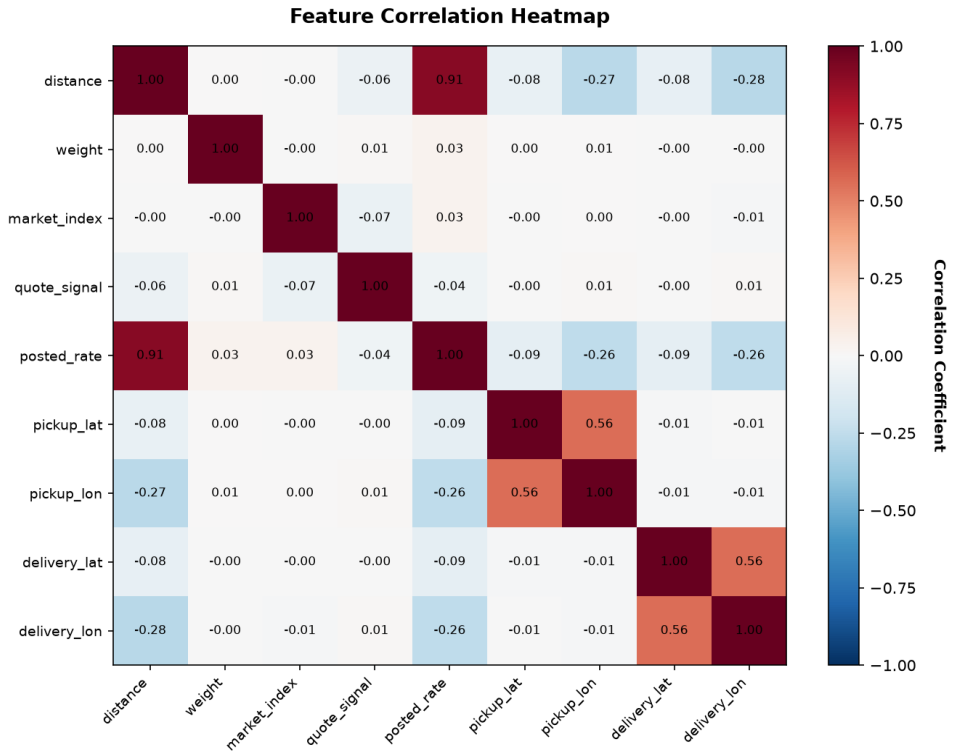


Figure 7: Correlation heatmap between key features

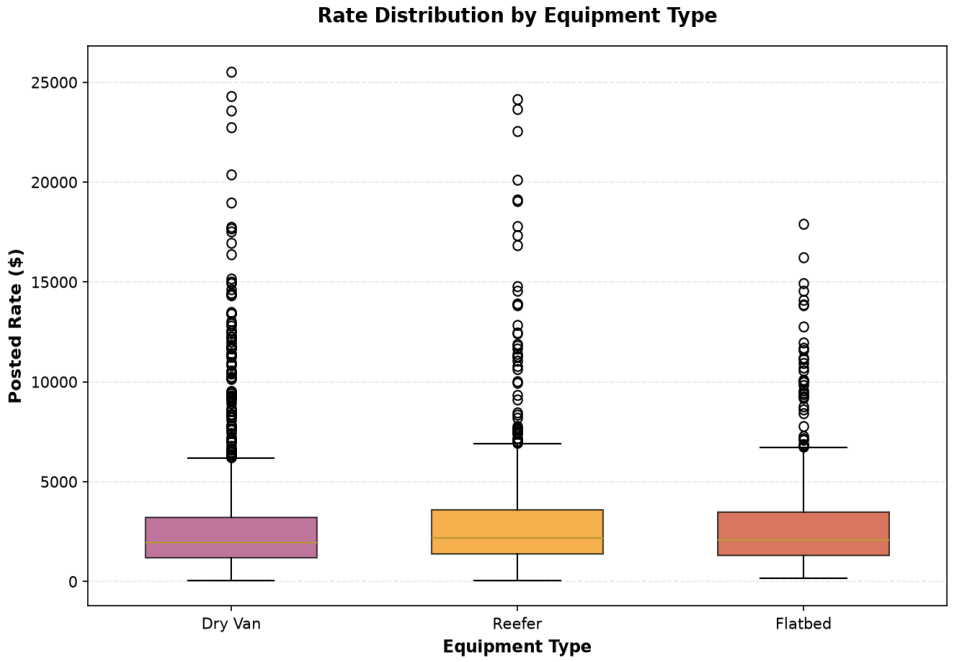


Figure 8: Rate distribution by equipment type

December Predictions

The model was used to predict rates for December 2025 with fixed parameters: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lbs. Only the date varies across the 31 predictions.

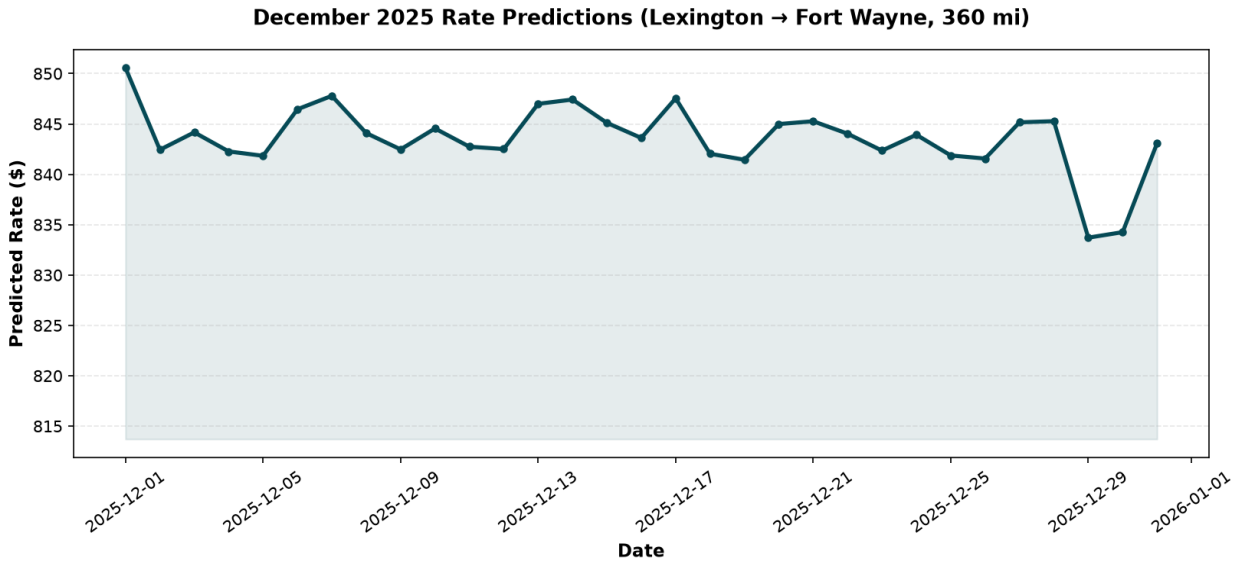


Figure 9: December 2025 rate predictions with fixed route parameters

Implementation

The solution uses Python with pandas for data processing, scikit-learn for modeling, and joblib for model persistence. The pipeline trains on 80% of data and validates on 20%. Missing values are filled with median. The model and encoder are saved for inference.

Key Decisions

Tree-based models over linear regression because freight pricing has clear non-linear patterns. Polynomial distance features help because longer trips don't scale linearly with cost. Interaction features capture that pricing depends on market conditions differently based on distance and weight.

Ensemble approach reduces variance and combines the strengths of both models. Using inverse MAE for weighting is simple but effective—gives more influence to the model that performs better.